

Block 4

webMethods Multi-Container Stacks

MSR · Universal Messaging · PostgreSQL · Startup order · Readiness

A Minimal compose.yaml for MSR

compose.yaml

```
services:
  msr:
    image: ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3
    ports: ["5555:5555"]
    environment:
      ADMIN_PASSWORD: ${ADMIN_PASSWORD:?set it in .env}
      LAB_ENV: dev
    volumes:
      - ./application.properties:\
        /opt/webmethods/IntegrationServer/application.properties:ro
    healthcheck:
      test: ["CMD-SHELL",
             "curl -fs http://localhost:5555/health/readiness || exit 1"]
      start_period: 60s
    networks: [backend]
networks:
  backend: {}
```

Top-level keys that matter: services, networks, volumes, secrets. Indentation is the structure: two spaces, never tabs.

Classification - Internal

Universal Messaging in Compose: Facts

Item	Value
Image	ibmwebmethods.azurecr.io/universalmessaging-server:12.1.0.0 (UID 1724)
Ports	9000 nsp/nhp realm port, 9200 Prometheus exporter; no web console in the container
Health	GET http://localhost:9000/health/ → {"ServerStatus":"OK"} (curl is in the image)
Data	/opt/webmethods/UniversalMessaging/server/umserver/data (also logs/)
Tuning	REALM_NAME, INIT_JAVA_MEM_SIZE, MAX_JAVA_MEM_SIZE (MB)
Admin	runUMTool.sh ListChannels / GetQueueInfo / PeekQueue inside the container; Enterprise Manager → nsp://localhost:9000

Classification - Internal

🔧 Hands-On: Exercise 4.1

4.1

UM alone: healthy in ~15 s, /health/ JSON, runUMTool CreateQueue / ListChannels, queue survives down/up

📁 Files

block4/exercises/um-only/compose.yaml

Participant Workbook, Block 4 · ~15 min · 🏁 WarmupQueue still listed after down + up

Classification - Internal

Connecting MSR to UM (application.properties)

application.properties

```
# native UM messaging (publish/subscribe triggers)
messaging.IS_UM_CONNECTION.um_rname=$env{UM_URL}        # nsp://um:9000
messaging.IS_UM_CONNECTION.enabled=true

# JNDI provider + JMS connection alias on UM
jndi.LAB_UM_JNDI.providerURL=$env{UM_URL}
jndi.LAB_UM_JNDI.initialContextFactory=com.pcbsys.nirvana.nSpace.NirvanaContextFactory
jms.LAB_UM_JMS.jndi_jndiAliasName=LAB_UM_JNDI
jms.LAB_UM_JMS.jndi_connectionFactoryLookupName=LabConnectionFactory
jms.LAB_UM_JMS.jndi_automaticallyCreateUMAdminObjects=true
jms.LAB_UM_JMS.clientID=$env{JMS_CLIENT_ID}               # unique per node!
jms.LAB_UM_JMS.enabled=true
```

UM queue names must not contain dots (i.e lab.orders would be rejected, but LabOrders works).
UM client libraries (nClient.jar, nJMS.jar) are already in the MSR image.

Classification - Internal

🔧 Hands-On: Exercise 4.2

4.2	Stock MSR + UM wired only by application.properties: um becomes healthy, then msr starts
4.2	/health shows UMAliases.IS_UM_CONNECTION UP; edit WARMUP_GREETING, restart, no rebuild
📁 **Files**	block4/exercises/msr-um/compose.yaml + application.properties

Participant Workbook, Block 4 · ~25 min · 🏁 UMAliases UP; the new value appears in configurationvariables.log

Classification - Internal

MSR + Database: JDBC Pools and Drivers

- MSR keeps its internal functions (ISInternal, Xref, dashboard stats) on an embedded database by default
- Moving ISCoreAudit/Xref to an external DB needs schemas from the Database Component Configurator (no public container image)
- Business data: a JDBC pool alias (jdbc.<alias>.* in application.properties) plus a driver alias
- The DataDirect PostgreSQL alias in the image has no driver jar: add org.postgresql:postgresql to lib/jars/custom
- Driver alias = a small XML in config/jdbc/driver/, baked into the custom image
- Credentials via $env{DB_PASSWORD}; the database gets its own named volume

Note: Swisscom's real database may be Oracle or MS SQL: the pattern is identical, only the driver jar and URL change.

Classification - Internal

The Custom MSR Image for Capstone 4

block4/capstone/msr/Containerfile

```
ARG MSR_IMAGE=ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3
FROM ${MSR_IMAGE}
ARG PG_JDBC_VERSION=42.7.4
ARG MAVEN=https://repo1.maven.org/maven2/org/postgresql/postgresql
# rarely changes: JDBC driver from Maven Central + driver alias
ADD --chown=1724:1724 ${MAVEN}/${PG_JDBC_VERSION}/postgresql-${PG_JDBC_VERSION}.jar \
    /opt/webmethods/IntegrationServer/lib/jars/custom/
COPY --chown=1724:1724 block4/capstone/msr/jdbc/PostgreSQL_JDBC_Driver.xml \
    /opt/webmethods/IntegrationServer/config/jdbc/driver/
# changes most: the package
COPY --chown=1724:1724 packages/SwisscomLab \
    /opt/webmethods/IntegrationServer/packages/SwisscomLab
USER 1724
```

Build context is the repository root, so the shared packages/ folder is visible. No configuration, no secrets in the image.

Classification - Internal

Service Dependencies & Startup Order

PostgreSQL	Universal Messaging	Microservices Runtime	API Gateway (Day 3)
pg_isready healthcheck	/health/ healthcheck	waits: postgres + um healthy	waits: load balancer healthy

depends_on with condition: service_healthy gates each stage on a real healthcheck.

Measured: if UM is missing while MSR starts, the JMS alias stays DISABLED and readiness still says 200.

Classification - Internal

Healthcheck-Gated depends_on

compose.yaml

```
  um:
    image: ibmwebmethods.azurecr.io/universalmessaging-server:12.1.0.0
    healthcheck:
      test: ["CMD-SHELL",
             "curl -fs http://localhost:9000/health/ | grep -q ServerStatus.:.OK"]
      interval: 10s
      retries: 12
  msr:
    depends_on:
      postgres: { condition: service_healthy }
      um:       { condition: service_healthy }
    healthcheck:
      test: ["CMD-SHELL", "curl -fs http://localhost:5555/health/readiness || exit 1"]
      start_period: 60s
```

The single highest-leverage Compose pattern for webMethods stacks.

Classification - Internal

Readiness vs. Liveness: Measured in Capstone 4

Scenario	/health/liveness	/health/readiness	What happens
All services healthy	200	200	JDBC, JMS and UM indicators UP
UM stopped at runtime	200	503	JMS + UMAliases DOWN; recovers ~15–30 s after UM returns
MSR started without DB/UM	200	503	DB calls fail; JMS alias is disabled
...then DB/UM started, MSR not restarted	200	200 (!)	JDBC recovers, JMS alias stays disabled: silent failure

Healthchecks and load balancers must use readiness; gate startup with service_healthy to avoid row 4.

Classification - Internal

🏆 Capstone 4: MSR + UM + PostgreSQL

Objectives

115 min

- UM service (from 4.1) and the custom MSR image: JDBC driver, alias, package
- application.properties: JDBC pool, UM connection, JNDI + JMS alias
- msr service: service names (never localhost), depends_on healthy, readiness healthcheck
- up --build; all three healthy; verify.sh: "All checks passed."
- Admin UI: JDBC pool test, messaging, JMS alias; runUMTool ListChannels
- Break it: MSR first, dependencies later = silent JMS failure

Prerequisites

- Exercises 4.1–4.2 done
- block4/capstone/ with .env from .env.example
- Ports 5555 and 9000 free
- Reference: block4/solution/

Deliverable
compose.yaml, Containerfile, application.properties; verify.sh all green; notes on break-it steps 15, 17, 19.

Classification - Internal

Multi-Component Gotchas & Best Practices

Gotchas & pitfalls

- ✗ localhost in a connection URL inside a container means the container itself, not the UM/DB service
- ✗ A JMS alias that fails during MSR startup is not retried; it stays disabled
- ✗ UM's data directory not on a volume: messages lost on recreate
- ✗ restart: always hides a configuration error behind an endless restart loop

Best practices

- ✓ Service names in every URL: nsp://um:9000, jdbc:postgresql://postgres:5432/labdb
- ✓ service_healthy gating for every upstream dependency of MSR
- ✓ Dedicated named volumes for UM and the DB, backed up like any stateful system
- ✓ restart: on-failure:5 so a real misconfiguration surfaces

Classification - Internal

💬 Q&A and Discussion: Capstone 4

1. *Readiness returned 200 while the JMS alias stayed disabled: how would we notice that silent failure in Prod?*

2. *Every URL used a service name, never localhost: which of today's IS settings hard-code hostnames or IPs?*

3. *Swisscom uses Oracle / MS SQL: what changes in this stack, and how many DB connections per environment?*

Classification - Internal

Day 2 Recap

- ✓ Containerfiles: layer order, build context, multi-stage, build secrets
- ✓ Custom MSR images: package and driver baked in, configuration mounted
- ✓ Compose: services, networks, volumes, .env; config first when debugging
- ✓ MSR + UM + PostgreSQL wired by service names and application.properties
- ✓ depends_on: service_healthy prevents a silent startup failure; LBs use readiness

Coming up next: Day 3: 400+ packages, CI and scanning, 2 × MSR behind nginx and API Gateway, then secrets, rolling updates, Graylog and the road to production.

Classification - Internal